JMAN GROUP

Scrapy Training Guide

2025 Jan

1 Introduction to Scrapy

1.1 Recap of Previous Topics

- **Python**: Object-oriented, high-level programming language.
- **Requests Library**: Simplifies making HTTP requests in Python.
- **Regex Library**: Helps in pattern matching and extracting specific data from text.
- **Selenium & Playwright**: Used for browser automation, essential for handling dynamic websites.

1.2 Overview of Web Scraping

- **Definition**: Web scraping is the process of extracting data from websites, converting unstructured web content into structured data.
- **Applications**:
  - Business intelligence
  - Price monitoring
  - Data analysis and aggregation
  - Research

1.3 What is Scrapy?

- **Definition**: Scrapy is an open-source web crawling framework written in Python designed to extract and process data efficiently from websites.
- **Key Features**:
  - Fast and efficient asynchronous web scraping using Twisted.
  - Built-in support for requests, responses, and data storage.
  - Provides a structured way to build web crawlers (spiders).
  - Offers tools to handle various scraping challenges: pagination, cookies, error handling, and more.

1.4 Why Scrapy?

1. **Speed and Efficiency**:
   - Scrapy uses the asynchronous networking library **Twisted**, which allows sending multiple requests simultaneously, improving scraping speed significantly.
2. **Built-in Extensibility**:
   - Provides **middlewares**, **pipelines**, and **selectors**, which can be customized to handle various scraping tasks, from error handling to custom data processing.
3. **Comprehensive Data Extraction**:
   - Supports various data output formats like **JSON**, **CSV**, **XML**, making data collection and processing straightforward.
4. **Scalability**:
   - Scrapy can handle large-scale web scraping projects with distributed crawling and can manage a vast number of requests simultaneously.
5. **Error Handling**:
   - Built-in mechanisms for retrying failed requests, handling different response statuses, and logging errors.

- **Ideal for**:
  - **Large-scale** web scraping projects involving crawling entire websites or multiple pages.
    - **Example**: Scraping product data from an entire e-commerce site to monitor prices and availability.
  - **Automated data collection** for research, business intelligence, or machine learning.
    - **Example**: Collecting large datasets of news articles for sentiment analysis in a machine learning project.
  - Projects requiring **high-performance** scraping and scalable solutions.
    - **Example**: Aggregating job listings from multiple job boards to create a comprehensive job search platform.

1.5 When not to consider Scrapy?

- **Small Projects**: For simpler tasks, using **Requests** or **BeautifulSoup** might be more appropriate due to Scrapy's complexity.
- **Stealth Limitations**: Scrapy lacks the ability to mimic real browsers, unlike **Selenium** and **Playwright**, making it less suitable for websites with aggressive bot-detection mechanisms.
- **Dynamic Websites**: Heavy JavaScript-based websites are better handled by tools like Selenium/Playwright. Scrapy can be integrated with these, but the process is more involved.
- **Learning Curve**: Scrapy has a steeper learning curve compared to simpler scraping libraries.
- **Less suitable for**:
  - Simple, single page scraping tasks.
    - **Example**: Extracting the weather forecast from a single webpage.
  - JavaScript-heavy websites without using additional tools.
    - **Example**: Scraping dynamic content from a social media site that relies heavily on JavaScript for rendering.
    - **Note**: Scrapy can be integrated with Selenium or Playwright to handle JavaScript-heavy sites.
  - Small one-off scraping projects.
    - **Example**: Quickly extracting contact information from a single webpage for a one-time use.
  - Projects with minimal computational resources.
    - **Example**: To extract the currency exchange rate from a webpage.

1.6 Comparison of Web Scraping Tools

Feature/Tool	Scrapy	Requests	BeautifulSoup	Selenium	Playwright	Crawlee
Language	Python	Python	Python	Multiple (Python, Java, etc.)	Multiple (Python, Java, etc.)	JavaScript/ Python
Asynchronous	Yes (using Twisted)	No	No	No	Yes	Yes
Ease of Use	Moderate (steeper learning curve)	Easy	Easy	Moderate	Moderate	Moderate
Browser Automation	No (requires integration with Selenium/Playwright for JS-heavy sites)	No	No	Yes	Yes	Yes
Data Extraction	Built-in support for XPath and CSS selectors	No	Yes (with parsing HTML)	Yes (with browser interaction)	Yes (with browser interaction)	Yes (with browser interaction)
Data Storage	Built-in support for JSON, CSV, XML	No	No	No	No	No
Scalability	High (supports distributed crawling)	Low	Low	Low	Low	High
Error Handling	Built-in mechanisms for retries, status handling, and logging	No	No	No	No	Yes
Extensibility	High (middlewares, pipelines, selectors)	Low	Low	Moderate	Moderate	High
Speed	High (asynchronous requests)	Moderate	Moderate	Low (depends on browser speed)	Low (depends on browser speed)	High
Best For	Large-scale web scraping projects, automated data collection	Simple HTTP requests	Parsing HTML and extracting data	Interacting with dynamic websites	Interacting with dynamic websites	Large-scale web scraping projects

Feature/Tool	Scrapy	Requests	BeautifulSoup	Selenium	Playwright	Crawlee
Not Suitable For	Simple tasks, heavy JavaScript websites without additional tools	Complex scraping tasks	Large-scale scraping projects	High-performance scraping	High-performance scraping	Simple tasks

2 Pre-Requisites for Using Scrapy

- **Basic Python knowledge**.
- **Familiarity with HTML, CSS, and web scraping concepts**.
- **Understanding of HTTP protocols** and how requests/responses work.
- Experience with **Requests**, **BeautifulSoup**, and **XPath**/CSS selectors (helpful but not mandatory).

3 Scrapy Components

3.1 Spiders

- **Definition**: Spiders are classes that define how Scrapy will extract data from a specific website.
- **Main Components**:
  - `name`: Unique identifier for the spider.
  - `start_urls`: List of URLs where the spider will start crawling.
  - `parse()`: The main method that handles the response and extracts data.
  - **Example**:

```python
class MySpider(scrapy.Spider):
    name = 'myspider'
    start_urls = ['http://example.com']

    def parse(self, response):
        # Data extraction logic goes here
        pass
```

3.2 Selectors

- **Selectors** are used to extract data from web pages.
- **XPath** and **CSS selectors** are supported.
  - **CSS Selector Example**:

```python
response.css('div.quote::text').getall()
```

  - **XPath Example**:

```python
response.xpath('//div[@class="quote"]/text()').getall()
```

3.3 Items

- **Items** represent structured data extracted from web pages.
- Define custom item classes to structure the data.
  - **Example**:

```
class QuoteItem(scrapy.Item):
    text = scrapy.Field()
    author = scrapy.Field()
    tags = scrapy.Field()
```

3.4 Item Loaders

- **Item Loaders** enhance data extraction by cleaning and transforming data before storing it into an item.
  - **Example**:
    ```
    from scrapy.loader import ItemLoader
    loader = ItemLoader(item=QuoteItem(), response=response)
    loader.add_css('text', 'span.text::text')
    loader.add_css('author', 'small.author::text')
    item = loader.load_item()
    ```

3.5 Pipelines

- **Pipelines** process items after they are scraped (e.g., cleaning, storing, validation).
  - **Example Pipeline**:
    ```
    class CleanQuotesPipeline:
        def process_item(self, item, spider):
            item['text'] = item['text'].strip('“”')
            return item
    ```

3.6 Middlewares

- **Middlewares** allow you to modify requests and responses before they are sent or processed.
  - **Use Cases**: Proxy rotation, user-agent rotation, error handling.

---

4 Advanced Scrapy Features

4.1 Handling Requests and Responses

- Scrapy manages requests and responses using the **Request** class and the spider's `parse()` method.
  - **Example**:
    ```
    scrapy.Request(url='http://example.com', callback=self.parse)
    ```

4.2 Concurrency Management

- Scrapy sends multiple requests concurrently, making it fast and efficient.
- The **Scheduler** manages how requests are sent and processed.

4.3 Data Export

- Scrapy supports multiple data export formats like **JSON**, **CSV**, and **XML**.
  - **Example**:
    ```
    scrapy crawl myspider -o output.json
    ```

4.4 Scrapy Shell

- **Scrapy Shell** allows you to interactively test scraping commands and selectors.

- **Usage**:
  `scrapy shell 'http://quotes.toscrape.com/'`

4.5 Logging and Debugging

- Scrapy includes robust logging tools for debugging issues.
  - **Example**: Use `log.msg()` to log custom messages during scraping.

4.6 Handling JavaScript

- Scrapy can be integrated with **Selenium** or **Playwright** for scraping JavaScript-heavy sites.

4.7 Error Handling

- Use **try-except** blocks to handle common errors like network failures and invalid responses.

---

5 Scrapy Tutorial: Step-by-Step

Please complete the installation of Scrapy before proceeding with the tutorial REF

5.1 Creating Your First Scrapy Project

1. **Initialize Project**: (Enter in CLI after installation)
   `scrapy startproject myproject`

2. **Project Structure**:
   - **spiders/**: Contains spider files.
   - **items.py**: Defines data structure for items.
   - **middlewares.py**: Defines custom request/response handling logic.
   - **pipelines.py**: For processing scraped data.
   - **settings.py**: Configuration file.

5.2 Creating a Spider

1. **Generate Spider**:
   ```
   cd myproject
   scrapy genspider quotes quotes.toscrape.com
   ```

2. **Modify the Spider**: (Enter in the created file)
   ```
   class QuotesSpider(scrapy.Spider):
       name = "quotes"
       start_urls = ['http://quotes.toscrape.com/']

       def parse(self, response):
           for quote in response.css('div.quote'):
               yield {
                   'text': quote.css('span.text::text').get(),
                   'author': quote.css('small.author::text').get(),
                   'tags': quote.css('div.tags a.tag::text').getall(),
               }
   ```

5.3 Running a Spider

- **Command**:
  ```
  scrapy crawl quotes
  ```

5.4 Handling Pagination

- Modify the spider to follow pagination links:
  ```
  next_page = response.css('li.next a::attr(href)').get()
  if next_page:
      yield response.follow(next_page, self.parse)
  ```

---

6 Deploying Scrapy Spiders

6.1 Proxies for Scaling

- Proxies are essential for large-scale scraping to avoid IP bans. Implement **proxy rotation** for better scalability.

6.2 Scheduling Spiders

- Deploy spiders to cloud services like **Scrapyd** for running them on a schedule.

6.3 Deployment Methods

6.3.1 Scrapyd

- **Description**: Scrapyd is a service for running Scrapy spiders. It provides an API to schedule and manage spiders.
- **Pros**:
  - Easy to set up and use.
  - Provides a web interface for monitoring spiders.
  - Supports scheduling and running multiple spiders concurrently.
- **Cons**:
  - Requires additional setup and maintenance.
  - Limited to Scrapy projects.

6.3.2 Scrapy Cloud (by Zyte)

- **Description**: Scrapy Cloud is a cloud-based platform provided by Zyte (formerly Scrapinghub) for deploying and running Scrapy spiders.
- **Pros**:
  - No need to manage infrastructure.
  - Built-in support for rotating proxies and other advanced features.
  - Provides detailed analytics and monitoring.
- **Cons**:
  - Can be expensive for large-scale projects.
  - Limited customization compared to self-hosted solutions.

6.3.3 AWS Lambda

- **Description**: AWS Lambda is a serverless computing service that lets you run code without provisioning or managing servers.
- **Pros**:

  - Scales automatically with demand.
  - Pay only for the compute time you use.
  - Integrates well with other AWS services.
- **Cons**:
  - Limited execution time (15 minutes per invocation).
  - Requires packaging Scrapy project as a Lambda function.

6.3.4 Docker

- **Description**: Docker allows you to containerize your Scrapy project, making it easy to deploy on any platform that supports Docker.
- **Pros**:
  - Consistent environment across different deployment platforms.
  - Easy to scale and manage using container orchestration tools like Kubernetes.
  - Portable and can be deployed on any cloud provider.
- **Cons**:
  - Requires knowledge of Docker and container orchestration.
  - Additional overhead of managing Docker infrastructure.

6.4 Comparison Table

Deployment Method	Ease of Use	Scalability	Cost	Customization	Maintenance
Scrapyd	Easy	Moderate	Low	Moderate	Moderate
Scrapy Cloud	Very Easy	High	High	Low	None
AWS Lambda	Moderate	High	Pay-per-use	Low	Low
Docker	Moderate	High	Variable	High	Moderate

7 Ethical Considerations

7.1 Importance of Ethical Scraping

Ethical scraping is crucial to maintain the integrity of the web scraping community and to avoid legal and technical repercussions. Unethical scraping can lead to severe consequences for both the scraper and the target website.

7.2 Key Ethical Practices

- **Respect robots.txt**: Ensure your spiders respect websites' `robots.txt` files.
- **Avoid Overloading Servers**: Use rate limiting to avoid sending too many requests too quickly.
- **Legal Considerations**: Always review and respect the terms of service of the website you're scraping.

7.3 Risks of Unethical Scraping

1. **Legal Consequences**:

  - **Example**: A company scraping data from a website without permission could face lawsuits for violating the website's terms of service or intellectual property rights.
  - **Risk**: Legal actions can result in hefty fines, injunctions, or even criminal charges.
2. **IP Bans and Blocking**:
  - **Example**: Sending too many requests in a short period can lead to the scraper's IP address being banned or blacklisted by the target website.
  - **Risk**: This can disrupt the scraping operation and require additional resources to circumvent the bans, such as using proxies.
3. **Server Overload and Denial of Service**:
  - **Example**: Overloading a server with excessive requests can cause the website to slow down or crash, leading to a denial of service for legitimate users.
  - **Risk**: This can damage the scraper's reputation and lead to potential legal actions from the affected website.
4. **Data Integrity Issues**:
  - **Example**: Scraping data without proper validation and error handling can result in collecting inaccurate or incomplete data.
  - **Risk**: This can lead to incorrect analysis and decision-making based on faulty data.
5. **Reputation Damage**:
  - **Example**: Being publicly identified as an unethical scraper can harm the reputation of the individual or organization.
  - **Risk**: This can lead to loss of trust from clients, partners, and the broader community.

7.4 Best Practices for Ethical Scraping

- **Respect robots.txt**: Always check and adhere to the `robots.txt` file of the target website.
- **Implement Rate Limiting**: Use rate limiting to ensure your requests do not overwhelm the target server.
- **Seek Permission**: Whenever possible, seek permission from the website owner before scraping.
- **Use Proxies Responsibly**: If using proxies, ensure they are used ethically and do not violate the target website's terms of service.
- **Validate and Clean Data**: Implement proper validation and cleaning mechanisms to ensure the accuracy and integrity of the scraped data.

---

8 Practical Projects and Use Cases

8.1 Use Cases

- **E-commerce Data Extraction**: Scrape product prices, names, and reviews.
- **Price Monitoring**: Track price changes over time.
- **Content Aggregation**: Collect articles from news websites.
- **Job Listing Scraper**: Gather job postings and their details for aggregation.

- **Customer Sentiment Analysis**: Extract and analyze customer reviews, comments, and replies to gauge sentiment.
- **Competitor Analysis**: Scrape competitor websites to gather data on their products, pricing, and promotions.
- **Market Assessment**: Collect data from various sources to assess market trends and demands.
- **Data Enrichment**: Enhance existing datasets by scraping additional information from various websites.
- **Real Estate Listings**: Scrape property details, prices, and locations from real estate websites.
- **Social Media Monitoring**: Collect posts, comments, and engagement metrics from social media platforms.
- **News Aggregator**: Scrape headlines and URLs from a news website.

---

9 Best Practices

- Use **rotating proxies** and **user agents** to avoid IP bans.
- Follow a **modular spider design** to keep your code maintainable.
- **Respect website policies** and terms of service when scraping.
- **Regularly test and monitor** your spiders for performance and accuracy.
- **Optimize XPath and CSS selectors**: Use efficient selectors to minimize the time spent parsing HTML.
- **Limit concurrent requests**: Adjust the `CONCURRENT_REQUESTS` setting to balance speed and server load.
- **Use caching**: Implement caching mechanisms to avoid redundant requests and speed up scraping.
- **Handle exceptions gracefully**: Use try-except blocks to manage errors and prevent your spider from crashing.
- **Minimize data processing during scraping**: Offload data processing to pipelines or external systems to keep spiders lightweight.
- **Use Scrapy's built-in extensions**: Leverage Scrapy's extensions for logging, stats collection, and more to enhance your scraping process.
- **Profile and benchmark your spiders**: Regularly profile your spiders to identify and address performance bottlenecks.
- **Optimize item pipelines**: Ensure that item pipelines are efficient and do not become a bottleneck in the scraping process.
- **Use asynchronous requests**: Take advantage of Scrapy's asynchronous capabilities to improve scraping speed and efficiency.
- **Save the inputs and outputs**: Keep a record of the URLs, requests, and responses and screenshots to debug and analyze the scraping process.

---

10 References and Resources

- Scrapy Official Website
- Scrapy Documentation
- Scrapy Playbook
- scrapy - GitHub

JMAN GROUP